

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION
TECHNOLOGY

COURSE PROJECT REPORT

Face Liveness Detection and Security Evaluation
Against Presentation Attacks

GROUP [Group number]

Nguyễn Quang Trung 20225557

Lê Thành An 20225562

Hạ Nhật Duy 20225544

Supervisor: Ph.D. Trần Nguyên Ngọc

Department: [Department]

School: School of Information and Communication Technology

HANOI, 06/2026

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION
TECHNOLOGY

COURSE PROJECT REPORT

Subject: Biometrics — Semester 2026.2

Face Liveness Detection and Security Evaluation Against Presentation Attacks

GROUP 5

Ha Nhat Duy	20225544
Nguyen Quang Trung	20225557
Le Thanh An	20225562

Supervisor: [Lecturer's name] _____

Signature

Department: [Department]

School: School of Information and Communication Technology

HANOI, 06/2026

ACKNOWLEDGMENTS

We would like to express our sincere gratitude to our supervisor, [Lecturer's name], for the guidance, constructive feedback, and encouragement provided throughout this project. The lectures and discussions in the Biometrics course gave us the theoretical foundation on which this work is built.

We are also grateful to the authors of the CASIA-FASD, CelebA-Spoof, and LCC-FASD datasets, whose publicly released data made our cross-dataset experiments possible, and to the open-source communities behind PyTorch, FastAPI, and React. Finally, we thank our families and friends for their patience and support during the completion of this report.

The authors

ABSTRACT

Face recognition is among the most convenient biometric modalities, yet it remains vulnerable to presentation attacks in which an adversary places a printed photograph or replays a video of a legitimate user in front of the camera. A recognition system that only verifies identity cannot tell a living person from such an artefact, so an explicit liveness check is required before the identity decision can be trusted. Existing defences range from costly hardware sensors to software methods based on hand-crafted texture features, the latter often failing to generalise across capture conditions.

This project develops a lightweight, real-time face liveness detection system that relies only on an ordinary webcam. We adopt a passive, single-frame approach based on a convolutional neural network because it requires no user co-operation and no special sensor. The chosen backbone is EfficientNet-B3, pre-trained on ImageNet and adapted to a two-class Live/Spoof problem through a two-phase transfer-learning procedure. To reduce the domain-shift problem, the model is trained on a cross-dataset mixture of CelebA-Spoof, CASIA-FASD, and LCC-FASD, with class imbalance handled by weighted sampling and a class-weighted loss. The trained model is served by an asynchronous FastAPI WebSocket backend and consumed by a React web client that streams webcam frames and displays the verdict in real time.

The main contribution of the project is a complete, working pipeline together with an honest security evaluation. On a held-out cross-dataset test set of 83,514 images, the model attains 95.83% accuracy, 97.0% recall, and an F1-score of 88.9%, corresponding to a false acceptance rate of about 4.4% and a false rejection rate of about 3.0%. We analyse this usability–security trade-off, identify the residual vulnerability of a purely texture-based classifier, and outline concrete improvements such as face cropping, blink-based active liveness, and threshold calibration.

The authors

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION.....	1
1.1 Motivation	1
1.2 Objectives and scope of the project	1
1.3 Tentative solution	2
1.4 Thesis organization.....	3
CHAPTER 2. REQUIREMENT SURVEY AND ANALYSIS.....	4
2.1 Status survey	4
2.2 Functional overview.....	5
2.2.1 General use case diagram.....	5
2.2.2 Business process	6
2.3 Functional description.....	6
2.3.1 Use case “Perform real-time liveness detection”.....	7
2.3.2 Use case “Start real-time session”.....	7
2.3.3 Use case “View verdict and confidence”	7
2.3.4 Use case “Submit single image for check”	7
2.4 Non-functional requirements	8
CHAPTER 3. THEORETICAL BACKGROUND AND TECHNOLOGIES.....	9
3.1 Presentation attacks and liveness cues.....	9
3.2 Convolutional neural networks and EfficientNet	9
3.3 Transfer learning	10
3.4 Data augmentation and class-imbalance handling	10
3.5 Technologies for the real-time system	11
3.6 Evaluation metrics	12

CHAPTER 4. DESIGN, IMPLEMENTATION, AND EVALUATION.....	14
4.1 Architecture design.....	14
4.1.1 Software architecture selection	14
4.1.2 Overall design.....	14
4.1.3 Detailed package design	15
4.2 Detailed design.....	15
4.2.1 User interface design.....	15
4.2.2 Class design and message flow.....	15
4.2.3 Application data management	18
4.3 Application building	18
4.3.1 Libraries and tools	18
4.3.2 Achievement.....	18
4.3.3 Illustration of main functions	19
4.4 Testing.....	20
4.5 Deployment	20
CHAPTER 5. SOLUTION AND CONTRIBUTION	22
5.1 Cross-dataset training of the liveness model	22
5.1.1 Problem	22
5.1.2 Solution	22
5.1.3 Result	23
5.2 Real-time serving design	24
5.2.1 Problem	24
5.2.2 Solution	24
5.2.3 Result	24
5.3 Security evaluation and analysis.....	24
5.3.1 Problem	24

5.3.2 Solution	25
5.3.3 Result	25
CHAPTER 6. CONCLUSION AND FUTURE WORK	28
6.1 Conclusion.....	28
6.2 Future work.....	29
REFERENCE	31
APPENDIX.....	33
A. TRAINING CONFIGURATION AND DATASET DETAILS	33
A.1 Preprocessing and augmentation	33
A.2 Training environment	33
A.3 Per-epoch training log	33
B. API AND WEBSOCKET MESSAGE SCHEMAS	35
B.1 Request–response endpoint	35
B.2 WebSocket message schemas	35

LIST OF FIGURES

Figure 2.1	General use-case diagram of the liveness detection system. . .	6
Figure 2.2	Activity diagram of the end-to-end liveness verification process.	6
Figure 4.1	Package decomposition and dependencies of the system. . .	15
Figure 4.2	Sequence of messages for a single real-time prediction. . . .	17
Figure 4.3	The system classifying a genuine face as live.	19
Figure 4.4	The system classifying a printed photo as a spoof.	20
Figure 5.1	Training loss and accuracy across the feature-extraction and fine-tuning phases.	23
Figure 5.2	Confusion-matrix heat-map on the cross-dataset test set. . . .	26

LIST OF TABLES

Bảng 2.1	Comparison of liveness-detection approaches against the project constraints.	5
Bảng 4.1	Languages, frameworks, and libraries used in the project. . . .	18
Bảng 4.2	Source-code statistics of the delivered system.	19
Bảng 5.1	Composition of the cross-dataset training set.	22
Bảng 5.2	Two-phase transfer-learning hyperparameters.	23
Bảng 5.3	Composition of the cross-dataset test set.	25
Bảng 5.4	Classification performance on the cross-dataset test set.	25
Bảng 5.5	Confusion matrix on the cross-dataset test set.	26
Bảng 5.6	Security metrics derived from the confusion matrix.	26
Bảng A.1	Phase 1 (feature extraction) per-epoch training log.	33
Bảng A.2	Phase 2 (fine-tuning) per-epoch training log.	34

CHAPTER 1. INTRODUCTION

1.1 Motivation

Biometric authentication has become a routine part of everyday digital life. Among the available modalities, the human face is the most convenient one: it can be acquired at a distance, without physical contact, and using nothing more than the camera already built into laptops, phones, and access-control gates. As a consequence, face recognition is now widely deployed for device unlocking, electronic know-your-customer (e-KYC) onboarding in banking, and physical access control.

This convenience, however, comes with a structural weakness. Unlike a fingerprint or an iris pattern, a person's face is publicly observable: it appears on social networks, identity cards, and video calls, and can therefore be captured by anyone. An attacker who obtains an image or a short video of a target can present that artefact to the camera and attempt to impersonate the victim. Such attempts are known as presentation attacks, and the printed photograph and the screen or video replay are by far their most common and lowest-cost forms.

The core of the problem is that a recognition system which only verifies identity treats a photograph of a user exactly as it treats the real user, because both produce a similar face representation. Without an additional safeguard, the system has no way of answering a more basic question that precedes identity: is the face in front of the camera a real, living person at all? For applications that guard money, personal data, or physical premises, an affirmative and reliable answer to that question is a prerequisite for trust. The need for a defence that is at the same time accurate, fast enough for interactive use, and deployable on commodity cameras therefore motivates the work presented in this report.

1.2 Objectives and scope of the project

Research and industry have proposed many countermeasures against presentation attacks. They can be divided broadly into hardware-based methods, which add specialised sensors such as infrared, depth, or thermal cameras, and software-based methods, which analyse only the ordinary colour image. Hardware methods are robust but increase cost and cannot be applied to the large installed base of plain webcams. Early software methods relied on hand-crafted texture descriptors, which are computationally cheap but tend to fail when the lighting, camera, or attack medium differs from those seen during development. More recent software methods use deep convolutional neural networks that learn discriminative spoof-

ing cues directly from data and currently provide the best accuracy on commodity cameras, at the cost of requiring large and varied training sets to generalise well.

The central limitation that remains is generalisation across capture conditions: a model trained on a single dataset frequently degrades sharply on images from a different camera or environment. Building on this observation, the project sets the following objectives. First, to build a lightweight convolutional model that classifies a face image as live or spoof. Second, to improve its cross-condition robustness by training on a mixture of several public datasets rather than a single one. Third, to integrate the model into a real-time web application whose main functions are to capture a webcam stream, transmit frames to the server, and display a live or spoof verdict together with a confidence value. Fourth, to evaluate the system not only by classical accuracy but also in standard biometric security terms and to analyse its residual vulnerabilities.

The scope is deliberately limited to two-dimensional presentation attacks, namely printed photographs and screen or video replays, which dominate in practice. Three-dimensional mask attacks and digital injection attacks are not addressed. The demonstrator targets a single-user, local-development deployment rather than a hardened production service. The detailed solution and its evaluation are presented in the later chapters, in particular in Chapter 5.

1.3 Tentative solution

The project approaches liveness detection as a passive, single-frame image-classification problem solved by transfer learning. The approach is passive because it requires no action from the user, and single-frame because each decision is made from one image, which keeps the latency low enough for real-time interaction. Transfer learning is chosen because training a deep network from scratch would require far more data and computation than are available for this project.

Concretely, the solution fine-tunes an EfficientNet-B3 network, pre-trained on ImageNet, into a two-class live-versus-spoof classifier, and trains it on a cross-dataset combination of three public face anti-spoofing corpora to encourage generalisation. The trained model is then exposed through an asynchronous web service and driven by a browser client that streams webcam frames and renders the verdict in real time. EfficientNet-B3 is selected for its strong accuracy-to-cost ratio, and the WebSocket-based client-server design is selected for its low-latency, bidirectional streaming, both of which directly serve the real-time requirement.

The main contribution of the project is a complete and reproducible pipeline, from data preparation and model training to a working real-time demonstrator, ac-

accompanied by an evaluation expressed in biometric security metrics. On a cross-dataset test set the model reaches an accuracy of 95.83% with a false acceptance rate of about 4.4%, and the demonstrator returns verdicts at roughly two frames per second with sub-second latency on a development machine.

1.4 Thesis organization

The remainder of this report is organised as follows.

Chapter 2 surveys the current state of face liveness detection. It reviews existing hardware- and software-based products and academic methods, compares their strengths and weaknesses, and from that analysis derives the functional and non-functional requirements of the system to be built, expressed through use cases.

Chapter 3 introduces the theoretical background and the technologies used in the project. It explains presentation attacks and their cues, convolutional neural networks and the EfficientNet family, the transfer-learning strategy, and the evaluation metrics, and it justifies each technical choice against the alternatives.

Chapter 4 presents the design, implementation, testing, and deployment of the system. It describes the client–server architecture, the backend and frontend modules, the libraries and tools used, the functional testing performed, and how the application is deployed.

Chapter 5 details the principal contributions of the project: the cross-dataset training methodology for the EfficientNet-B3 model, the real-time WebSocket serving design, and the security evaluation, including the confusion-matrix analysis and the discussion of the usability–security trade-off.

Chapter 6 concludes the report by summarising what was achieved and what was not, comparing the result with similar systems, and proposing directions for future work.

CHAPTER 2. REQUIREMENT SURVEY AND ANALYSIS

This chapter analyses the context in which the system operates and derives its requirements. Section 2.1 surveys existing liveness-detection solutions and compares their characteristics. Section 2.2 summarises, at a high level, the functions the system must provide, using a use-case model and the end-to-end verification process. Section 2.3 specifies the most important use cases in detail, and Section 2.4 states the non-functional requirements.

2.1 Status survey

A clear picture of the requirements is obtained by examining three sources: the needs of the end user, comparable systems already in operation, and similar academic or commercial solutions. From the user’s perspective, the desired behaviour is simple to state. When a face is shown to the camera, the system should quickly and reliably indicate whether that face belongs to a living person or to a spoofing medium such as a printed photograph or a replayed video, and it should do so without imposing tedious actions on a genuine user.

Comparable systems differ mainly in the type of evidence they exploit. Hardware-based solutions, such as the depth-sensing face unlock found on some smartphones, project and read back a structured-light or infrared pattern to reconstruct facial geometry. They are highly resistant to flat photo attacks, but they depend on dedicated sensors that ordinary webcams do not possess. Software-based solutions work from the colour image alone. Within this group, a first family relies on hand-crafted texture and image-quality features, for example local binary patterns applied in different colour spaces [1]; these methods are fast but tend to lose accuracy when the camera or environment changes. A second, more recent family uses deep convolutional neural networks, optionally with auxiliary depth or remote-pulse supervision [2], and currently offers the best accuracy on commodity cameras at the cost of needing large and diverse training data [3]. Active methods form a third group: they ask the user to blink, turn the head, or smile, which adds robustness against static photos but introduces friction and still fails against video replays.

Table 2.1 compares these approaches along the dimensions that matter for this project, namely the sensor required, the robustness against the targeted attacks, the user effort, and the suitability for a low-cost real-time deployment. The comparison shows that a passive, software, CNN-based approach offers the best balance for the project’s constraints: it needs only a webcam, requires no user action, and can reach high accuracy, while its main weakness, the dependence on diverse training data,

can be mitigated by cross-dataset training. This finding directly shapes the solution adopted in the following chapters.

Approach	Sensor	Robustness to 2D attacks	User effort	Real-time on webcam
Hardware (IR/depth)	Special	High	None	No (needs sensor)
Hand-crafted texture	RGB only	Medium	None	Yes
Active (blink/-motion)	RGB only	Medium–High	High	Yes
Deep CNN (passive)	RGB only	High	None	Yes

Table 2.1: Comparison of liveness-detection approaches against the project constraints.

Based on this survey, the system to be developed should provide, as its essential features, the real-time capture of webcam frames, the classification of each frame as live or spoof by a CNN model, and the immediate presentation of the verdict and its confidence to the user. A secondary feature is a single-image checking interface for testing and integration purposes.

2.2 Functional overview

This section summarises the functions of the software at a high level. Detailed specifications of the selected use cases are deferred to Section 2.3.

2.2.1 General use case diagram

The system has a single human actor, the User, who is simultaneously the person being verified and the operator of the demonstrator. The backend model can be regarded as a supporting actor that performs the classification. Figure 2.1 shows the general use-case diagram. The User interacts with four use cases: starting a real-time session, which includes granting camera access and establishing the connection; running the real-time liveness detection, which streams frames and is the core function; viewing the verdict and confidence; and, separately, submitting a single image for a one-off check. The latter exercises the same model through a request–response interface rather than the streaming interface.

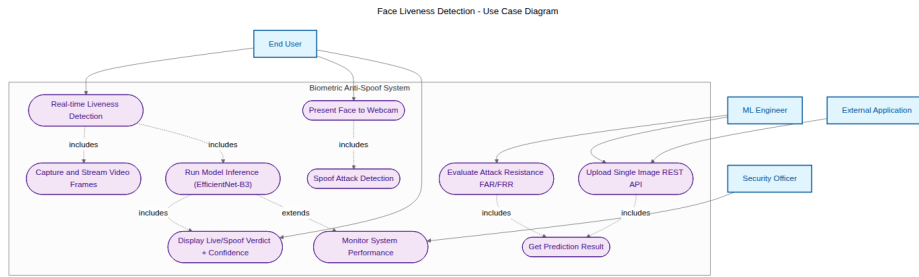


Figure 2.1: General use-case diagram of the liveness detection system.

2.2.2 Business process

The use cases above combine into a single end-to-end verification process. The User opens the web application and grants camera permission; the client establishes a streaming connection to the server; from that moment the client samples a frame at a fixed interval and sends it to the server; the server decodes each frame, runs the model, and returns a verdict; and the client renders the result, updating continuously as new frames arrive. The process ends when the User closes the page or the connection is terminated. This activity flow is illustrated in Figure 2.2 and is realised technically by the architecture described in Chapter 4.

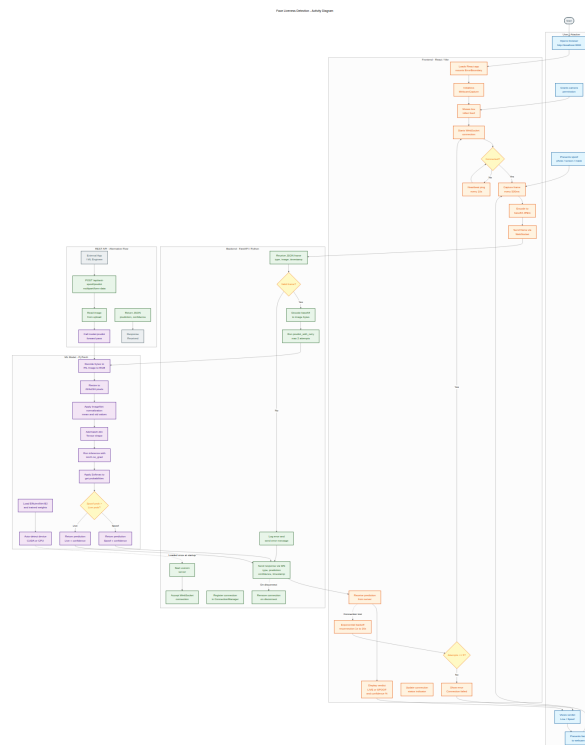


Figure 2.2: Activity diagram of the end-to-end liveness verification process.

2.3 Functional description

This section specifies the four most important use cases. For each, the actor, pre-conditions, main flow, alternative flow, and post-conditions are given.

2.3.1 Use case “Perform real-time liveness detection”

This is the core use case. The actor is the User. The pre-condition is that the camera permission has been granted and the streaming connection to the server is open. In the main flow, the client captures a frame from the webcam every 500 milliseconds, encodes it, and sends it to the server; the server decodes the frame, runs the model, and returns the predicted class together with a confidence score; the client displays the result. The flow repeats continuously. In the alternative flow, if a frame cannot be decoded or the model fails, the server returns an error message describing the cause and the session continues with the next frame, so that a single bad frame does not interrupt detection. The post-condition is that the most recent verdict and its confidence are shown to the User.

2.3.2 Use case “Start real-time session”

The actor is the User. The pre-condition is that the application page has been loaded in a browser that exposes a camera. In the main flow, the application requests camera permission; once granted, it opens the streaming connection and indicates a connected state. In the alternative flow, if the permission is denied, the application shows a message asking the User to enable the camera; if the connection cannot be established, the client retries automatically with increasing delays. The post-condition is that the system is ready to stream frames, or that an explanatory message is shown.

2.3.3 Use case “View verdict and confidence”

The actor is the User. The pre-condition is that at least one prediction has been received. In the main flow, the client displays a green “Live” indicator or a red “Spoof” indicator together with the confidence percentage, and updates this display each time a new prediction arrives. The post-condition is that the User can read the current liveness decision.

2.3.4 Use case “Submit single image for check”

The actor is the User or an integrating client. The pre-condition is that the back-end service is running and the model is loaded. In the main flow, the actor uploads an image file to the request–response endpoint; the server validates that the upload is an image, runs the model, and returns the verdict, the confidence, and the per-class probabilities. In the alternative flow, if the upload is not an image the server returns a client-error response, and if the model is not loaded it returns a service-unavailable response. The post-condition is that the classification result, or an explanatory error, is returned.

2.4 Non-functional requirements

Beyond the functions above, the system must satisfy several non-functional requirements. In terms of performance, the end-to-end latency from frame capture to verdict should remain below one second so that the interaction feels responsive, and the client should not overload the network or the server, which is why the sampling rate is limited to about two frames per second. In terms of security effectiveness, the false acceptance rate, that is the proportion of spoofs wrongly accepted, should be as low as possible while keeping the false rejection rate of genuine users at an acceptable level. In terms of reliability, the streaming connection should recover automatically from transient failures, and a malformed frame should never crash the service. In terms of usability, a genuine user should not be required to perform special actions, and the verdict should be presented clearly through colour and text. Finally, in terms of maintainability and portability, the backend and frontend should be separated by a well-defined interface, the model should be replaceable without changing the serving code, and the service should be containerisable for deployment.

This chapter has surveyed existing liveness solutions, concluded that a passive CNN-based approach best fits the project constraints, and derived the functional and non-functional requirements of the system. The theoretical foundations and technologies needed to satisfy these requirements are presented in the next chapter.

CHAPTER 3. THEORETICAL BACKGROUND AND TECHNOLOGIES

This chapter presents the theoretical foundations and the technologies on which the system is built. Each topic is linked to a requirement identified in Chapter 2, and, where several options exist, the alternatives are listed and the chosen one is justified. Section 3.1 explains presentation attacks and the cues that distinguish a live face from a spoof. Section 3.2 introduces convolutional neural networks and the EfficientNet family. Section 3.3 describes transfer learning, and Section 3.4 the techniques used to cope with limited and imbalanced data. Section 3.5 reviews the technologies used to build the real-time service, and Section 3.6 defines the evaluation metrics.

3.1 Presentation attacks and liveness cues

A presentation attack is an attempt to subvert a biometric system by presenting an artefact, called a presentation attack instrument, to the sensor. For face, the most common instruments are the printed photograph and the screen or video replay, both of which reproduce a two-dimensional image of the victim's face. The task of liveness detection, also called presentation attack detection, is to decide whether the captured face is a bona-fide presentation by a living person or an attack presentation.

Software-based detection is possible because re-capturing a face through a second medium leaves measurable traces. A printed photograph is flat, reflects light differently from skin, and may exhibit printing artefacts and a loss of fine texture. A screen replay introduces its own artefacts, such as the moiré pattern produced by photographing a pixel grid, an altered colour gamut, and the luminance of the backlight. In addition, a re-captured face lacks the natural three-dimensional micro-motion of a living face. A convolutional model trained on enough examples can learn to recognise this combination of texture and colour irregularities directly from the image, which is the property exploited in this project.

3.2 Convolutional neural networks and EfficientNet

A convolutional neural network is a model that applies learnable convolutional filters in a hierarchy of layers, transforming raw pixels into progressively more abstract features and ending in a classifier. Early layers respond to edges and textures, while deeper layers capture higher-level structure; this makes such networks well suited to the texture-driven cues described above.

EfficientNet [4] is a family of convolutional networks designed around com-

pound scaling. Instead of increasing only depth, only width, or only input resolution, compound scaling increases all three together according to a fixed ratio, which yields a better accuracy-to-cost trade-off than scaling any single dimension. The family ranges from B0, the smallest, to B7, the largest. This project uses the B3 variant, which has roughly twelve million parameters and offers a strong balance between accuracy and computational cost, in keeping with the real-time requirement of Section 2.4.

The choice of backbone was made among several candidates. MobileNetV2 [5] is lighter and would run faster, but it offers lower representational capacity, which is a disadvantage given the subtle nature of spoof cues. Deeper residual networks would provide more capacity but at a markedly higher inference cost that conflicts with the latency budget. EfficientNet-B3 was selected because its compound-scaled design attains the accuracy of much larger networks while remaining light enough to serve in near real time. It is worth noting that an earlier design considered MobileNetV2; the final implementation adopts EfficientNet-B3 for the reasons given here.

3.3 Transfer learning

Training a deep network from random initialisation requires very large datasets and substantial computation, neither of which is justified for this project. Transfer learning avoids this by starting from a network already trained on a large generic dataset and adapting it to the new task. The backbone used here is pre-trained on ImageNet [6], so its early layers already encode generic visual primitives such as edges, colours, and textures that transfer well to the spoof-detection task.

The adaptation follows a two-phase procedure. In the first phase, feature extraction, the convolutional backbone is frozen and only a newly attached two-class head is trained; this quickly aligns the output layer to the live-versus-spoof problem without disturbing the pre-trained features. In the second phase, fine-tuning, the entire network is unfrozen and trained further with a small learning rate, so that all layers gently specialise to spoofing cues. The alternative of training the whole network from scratch was rejected because it would need far more data to reach comparable accuracy and would be prone to overfitting on the available material.

3.4 Data augmentation and class-imbalance handling

Two practical problems accompany the data used in this project: the risk of overfitting to a particular capture condition, and the imbalance between the number of live and spoof images. Data augmentation addresses the first problem by applying randomised transformations to each training image, such as cropping, horizontal

flipping, rotation, colour jitter, blurring, and random erasing. These transformations force the network to rely on intrinsic spoof texture rather than on incidental framing or colour balance, improving robustness across conditions.

Class imbalance is addressed by two complementary techniques. A weighted random sampler draws training examples so that, on average, each class is represented equally within a minibatch, and a class-weighted loss function penalises errors on the minority class more heavily. Without these measures, a classifier trained on imbalanced data would be biased towards the majority class and would report misleadingly high accuracy while performing poorly on the minority class. The alternative of simply discarding majority-class samples to balance the set was rejected because it would waste useful data.

3.5 Technologies for the real-time system

The serving system is built on a small, well-supported technology stack, each element of which addresses a specific requirement.

The model is implemented and served with PyTorch and its companion library Torchvision [7], which provide the EfficientNet architecture, the ImageNet-pre-trained weights, and the image-preprocessing transforms. PyTorch was preferred over the alternative deep-learning framework TensorFlow mainly for the convenience of its eager execution and the direct availability of the chosen backbone.

The backend is built with FastAPI [8], an asynchronous Python web framework. FastAPI is used because it natively supports both conventional request–response endpoints and persistent WebSocket connections, and because its asynchronous model suits the concurrent handling of a frame stream. For the real-time channel, a WebSocket is preferred over repeated request–response calls: a WebSocket keeps a single connection open and allows the server to push results back to the client with low latency, whereas issuing a separate request for every frame would incur repeated connection overhead. The request–response endpoint is retained only for single-image testing.

The frontend is built with React [9] and the Vite development server, using the react-webcam component to access the camera. React was chosen for its component model, which keeps the capture logic, the connection logic, and the result display cleanly separated, and for the ease with which it manages the asynchronous updates produced by the incoming verdict stream.

3.6 Evaluation metrics

The system is evaluated both as a classifier and as a biometric security mechanism. Adopting the convention that the live class is the positive class, the four basic counts are the true positives TP (live faces classified as live), the true negatives TN (spoofs classified as spoof), the false positives FP (spoofs classified as live), and the false negatives FN (live faces classified as spoof). From these counts, the classification metrics are defined in Equations (3.1) to (3.4).

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (3.1)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (3.2)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3.3)$$

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3.4)$$

Accuracy measures overall correctness but can be misleading when the classes are imbalanced. Precision and recall describe, respectively, how trustworthy a “live” decision is and how many genuine live faces are accepted, and the F1-score is their harmonic mean.

For the security viewpoint, the relevant quantities follow the standard ISO/IEC 30107-3 reporting terminology [10]. The false acceptance rate, equivalent to the attack presentation classification error rate, is the fraction of spoofs wrongly accepted as live, given in Equation (3.5). The false rejection rate, equivalent to the bona-fide presentation classification error rate, is the fraction of live users wrongly rejected, given in Equation (3.6). Their average, the average classification error rate of Equation (3.7), provides a single balanced summary.

$$\text{FAR} = \text{APCER} = \frac{FP}{FP + TN} \quad (3.5)$$

$$\text{FRR} = \text{BPCER} = \frac{FN}{FN + TP} \quad (3.6)$$

$$\text{ACER} = \frac{\text{APCER} + \text{BPCER}}{2} \quad (3.7)$$

The false acceptance rate is the security-critical metric, because a high value means the system is easy to spoof, whereas the false rejection rate is the usability metric. Lowering one by moving the decision threshold necessarily raises the other,

a trade-off analysed in Chapter 5.

This chapter has established the theory of presentation attacks, justified the choice of an EfficientNet-B3 model fine-tuned by transfer learning, described the techniques used to handle limited and imbalanced data, motivated the FastAPI, WebSocket, and React technology stack, and defined the evaluation metrics. The next chapter applies these foundations to the design, implementation, and testing of the system.

CHAPTER 4. DESIGN, IMPLEMENTATION, AND EVALUATION

This chapter describes how the system is designed, implemented, tested, and deployed. Section 4.1 presents the software architecture and the decomposition into packages. Section 4.2 details the user interface, the principal classes, and the way application data is managed. Section 4.3 reports the tools used, the size of the resulting code base, and the main screens. Section 4.4 describes the testing performed, and Section 4.5 the deployment. The evaluation of the model as a biometric mechanism, being the principal contribution, is presented separately in Chapter 5.

4.1 Architecture design

4.1.1 Software architecture selection

The system adopts a client–server architecture, with each side internally organised in layers. This style is the natural fit for the problem, because the heavy computation, namely the neural network inference, must run on a capable server, while the user interface and camera access must run in the user’s browser. The client is responsible for capturing frames and presenting results; the server is responsible for decoding frames, running the model, and returning verdicts. The two sides communicate over a WebSocket channel, chosen in Section 3.5 for its low-latency streaming. Within the server, a clear separation is maintained between the web layer, which handles connections and message validation, and the model layer, which performs preprocessing and inference; within the client, the capture, communication, and display concerns are each handled by a dedicated component.

4.1.2 Overall design

Figure 4.1 shows the package-level decomposition and the dependencies between packages. On the server side, the entry-point package configures the application and cross-origin policy and includes the routing package; the routing package depends on two endpoint packages, one for the request–response interface and one for the WebSocket interface; both endpoint packages depend on the model-service package, which in turn depends on PyTorch and the stored weights. On the client side, the application package depends on three component packages for capture, communication, and display. The dependencies are strictly directed from the upper layers towards the lower layers, with no cycles, which keeps the model layer independent of the web layer and the display independent of the capture logic.

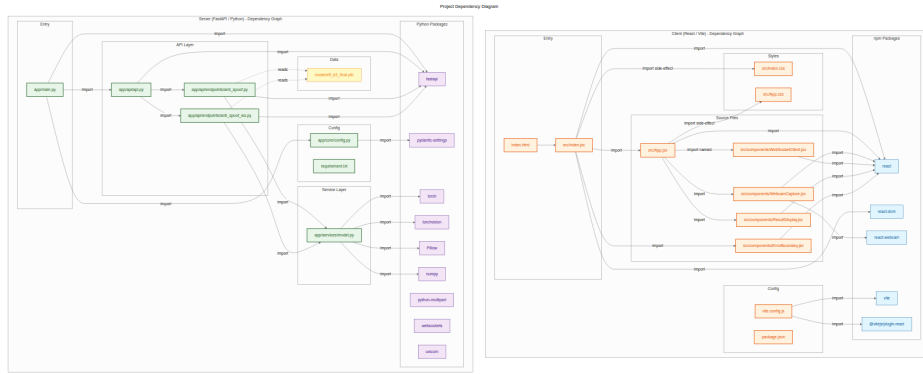


Figure 4.1: Package decomposition and dependencies of the system.

4.1.3 Detailed package design

The model-service package is the centre of the server design. It contains the class `AntiSpooofModel`, which encapsulates the network, the preprocessing transform, and the class labels. The two endpoint packages each hold a router that obtains image bytes, delegates the classification to `AntiSpooofModel`, and formats the response; the WebSocket endpoint additionally contains a connection-manager class that tracks open connections and sends messages. This arrangement means that the model can be replaced, or its preprocessing changed, without modifying any web-layer code, satisfying the maintainability requirement of Section 2.4.

4.2 Detailed design

4.2.1 User interface design

The application targets a desktop browser on a screen of at least 1280 by 720 pixels, with camera access. The interface follows a simple two-column convention: the live webcam preview is shown on the left and the verdict on the right, with a connection indicator in the header and a debug summary in the footer. The verdict uses a consistent colour code, green for a live face and red for a spooof, reinforced by a text label and an icon, so that the decision is legible at a glance. The confidence value is shown as a percentage beneath the verdict. Figure ?? sketches this layout.

4.2.2 Class design and message flow

The principal server class is `AntiSpooofModel`. On construction it rebuilds the EfficientNet-B3 graph with a two-output head, loads the trained weights onto the available device, and sets the network to evaluation mode; its `predict` method accepts the raw image bytes, applies the preprocessing transform, runs a forward pass, and returns the predicted label, the confidence, and the per-class probabilities. Listing 4.1 shows the essence of this method. The second important class, the connection manager of the WebSocket endpoint, accepts and tracks connections

and serialises outgoing messages as JSON.

```
1 def predict(self, image_bytes: bytes):
2     image = Image.open(io.BytesIO(image_bytes)).convert('RGB')
3     input_tensor = self.transform(image).unsqueeze(0).to(self.
4         device)
5     with torch.no_grad():
6         outputs = self.model(input_tensor)
7         probabilities = torch.softmax(outputs, dim=1)[0]
8         prob_list = probabilities.cpu().numpy().tolist()
9         pred_idx = int(np.argmax(prob_list))
10    return {
11        "prediction": self.classes[pred_idx],
12        "confidence": prob_list[pred_idx],
13        "probabilities": {self.classes[0]: prob_list[0],
14                           self.classes[1]: prob_list[1]},
15    }
```

Listing 4.1: Core of the inference method in `AntiSpooofModel`.

The interaction between the components during a single prediction is the following. The capture component produces a frame and passes it to the communication component, which sends it over the WebSocket; the server's WebSocket endpoint validates the message, asks `AntiSpooofModel` to classify the image, and returns the verdict; the communication component receives the verdict and hands it to the display component, which updates the screen. This sequence is illustrated in Figure 4.2.

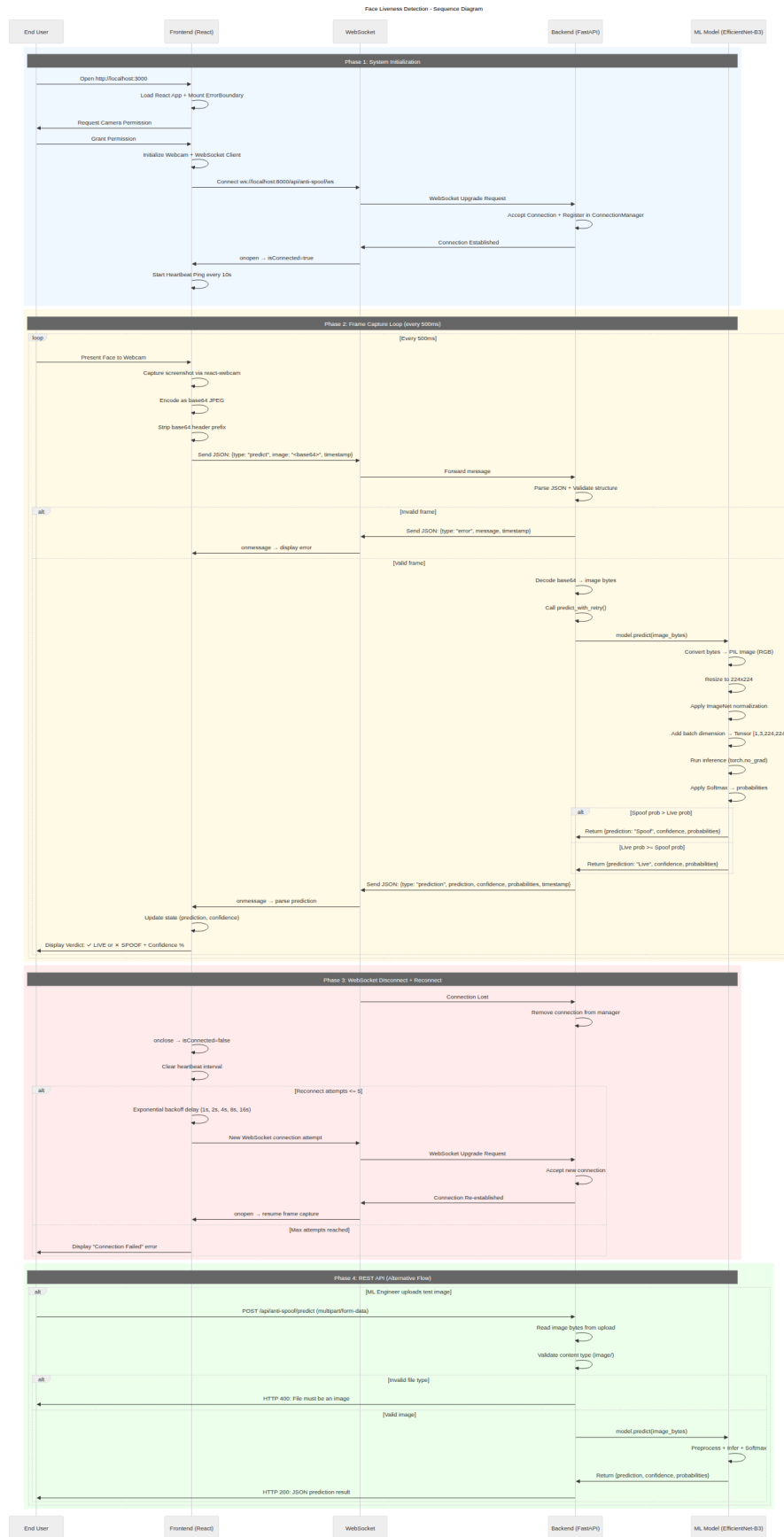


Figure 4.2: Sequence of messages for a single real-time prediction.

4.2.3 Application data management

The system is stateless and does not use a persistent database, because the demonstrator neither enrolls users nor stores verification history; each frame is classified independently and the result is transient. The only persistent artefacts are the trained model weights, stored as a single file of about 41.3 megabytes, and the image datasets used offline during training, whose composition is detailed in Appendix A. Persisting verification logs to a database is identified as future work in Chapter 6.

4.3 Application building

4.3.1 Libraries and tools

Table 4.1 lists the main languages, frameworks, and libraries used to build the system, together with their versions. The backend dependencies are taken from the project’s requirements file and the frontend dependencies from its package manifest.

Purpose		Tool / library	Version
Model training and inference		PyTorch, Torchvision	2.6 (CUDA 12.4)
Backend framework	web	FastAPI	latest
ASGI server		Uvicorn	latest
WebSocket support	sup-	websockets	latest
Image handling		Pillow, OpenCV-headless, NumPy	latest
Frontend library		React, React DOM	18.2.0
Webcam access		react-webcam	7.1.0
Frontend build tool	build	Vite, @vitejs/plugin-react	4.3.9 / 4.0.0
Containerisation		Docker	—
Training environment	environ-	Kaggle Notebook, Tesla P100 GPU	—

Table 4.1: Languages, frameworks, and libraries used in the project.

4.3.2 Achievement

The delivered product consists of two packaged components: a containerised FastAPI backend that serves the model, and a React frontend that can be served as static assets after building. Table 4.2 summarises the size of the source code. The backend comprises about 315 lines of Python across six modules, and the frontend about 469 lines of JavaScript and 367 lines of style sheets across six components,

giving a compact code base of roughly 1,150 lines in addition to the training notebook. The trained model file occupies about 41.3 megabytes.

Component	Files	Lines of code
Backend (Python)	6	315
Frontend (JavaScript/JSX)	6	469
Frontend (CSS)	2	367
Trained model weights	1	41.3 MB

Table 4.2: Source-code statistics of the delivered system.

4.3.3 Illustration of main functions

The two main situations the demonstrator must handle are a genuine face and a spoof. Figure 4.3 shows the application when a real face is presented, where the verdict panel turns green and reports a live decision, and Figure 4.4 shows it when a printed photograph or a phone screen is presented, where the panel turns red and reports a spoof. In both cases the confidence percentage and the connection status are visible, and the footer counter confirms that frames are being processed continuously.

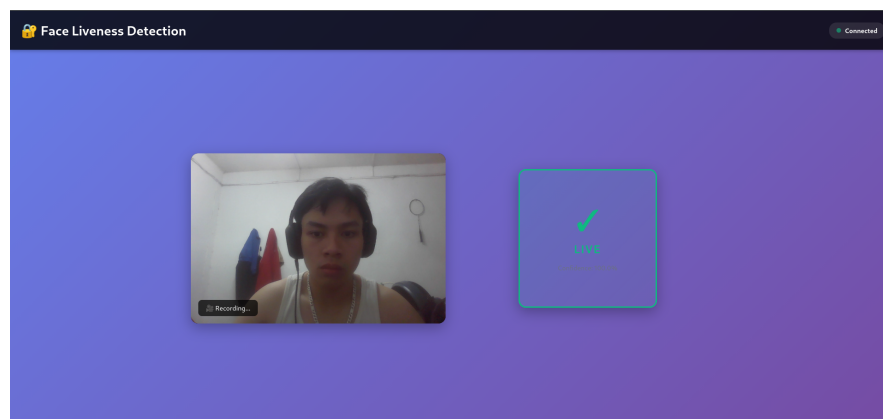


Figure 4.3: The system classifying a genuine face as live.

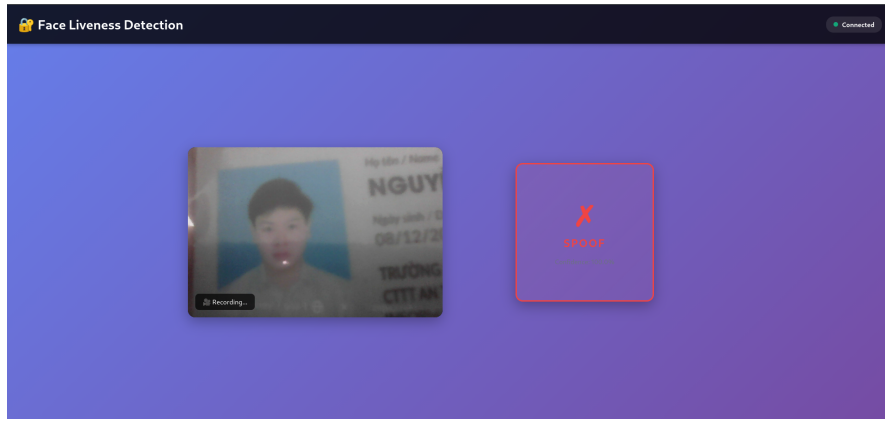


Figure 4.4: The system classifying a printed photo as a spoof.

4.4 Testing

The system was validated by functional, end-to-end testing of the two most important functions, namely real-time detection and single-image checking, complemented by robustness checks on the communication channel. For real-time detection, the application was run against a genuine face and against print and replay attacks, and the displayed verdict was compared with the expected outcome; the verdict updated continuously at the expected rate of about two frames per second. For single-image checking, images were submitted to the request–response endpoint and the returned verdict, confidence, and probabilities were inspected. Robustness was checked by sending malformed messages, such as invalid JSON or an absent image field, and by interrupting the connection; in each case the server returned a descriptive error and continued to operate, and the client re-established the connection automatically. The functional outcomes matched expectations. A limitation of this testing is that it was qualitative and conducted by the developers; a systematic, quantitative evaluation of detection quality is reported in Chapter 5, and a small number of robustness defects identified during testing are discussed in Chapter 6.

4.5 Deployment

During development, the backend runs locally under the Uvicorn server on port 8000 with automatic reload, and the frontend runs under the Vite development server on port 3000 with hot module replacement; the cross-origin policy of the backend is configured to accept the frontend origin. For wider deployment, the backend is packaged with the provided Dockerfile into a container image that can be run on a server equipped with a GPU for faster inference, and the frontend is compiled into static assets that can be served by any static host. A production deployment would, in addition, place the WebSocket behind a secure transport and

update the allowed origin to the production domain.

This chapter has presented the client–server architecture, the package and class design, the tools and size of the implementation, the functional testing, and the deployment of the system. The next chapter sets out the project’s principal contributions and the quantitative evaluation of the model.

CHAPTER 5. SOLUTION AND CONTRIBUTION

This chapter presents the three principal contributions of the project. The first is a cross-dataset training pipeline that produces a generalising EfficientNet-B3 liveness model. The second is a resilient, low-latency serving design that turns the model into a real-time service. The third is a security evaluation that quantifies the system’s effectiveness and exposes its residual weaknesses. Each contribution is presented in terms of the problem it solves, the solution adopted, and the result achieved, without repeating the descriptive material of the earlier chapters.

5.1 Cross-dataset training of the liveness model

5.1.1 Problem

The chief obstacle to a useful liveness model is poor generalisation across capture conditions. A model trained on a single dataset learns the specific cameras, lighting, and attack media of that dataset and degrades when confronted with new ones. A second obstacle is the imbalance between the number of live and spoof images available, which biases a naively trained classifier towards the majority class. The contribution described here is a training pipeline designed to overcome both obstacles.

5.1.2 Solution

The first element of the solution is to train on a mixture of three public face anti-spoofing datasets rather than one, so that the model is exposed to a wide range of cameras, scenes, and attack types. The datasets are CelebA-Spoof [11], CASIA-FASD [12], and LCC-FASD [13]. The composition of the resulting training set is given in Table 5.1; the label convention used throughout is that zero denotes a spoof and one denotes a live face. The training set contains 137,140 images.

Source	Live	Spoof	Subtotal
CelebA-Spoof (balanced sample)	35,547	35,547	71,094
CASIA-FASD (train split)	19,011	38,736	57,747
LCC-FASD (training split)	1,223	7,076	8,299
Total	55,781	81,359	137,140

Table 5.1: Composition of the cross-dataset training set.

The second element is the two-phase transfer-learning procedure motivated in Section 3.3, applied to an EfficientNet-B3 backbone whose final layer is replaced by a two-output linear head. In the first phase the backbone is frozen and only the head is trained; in the second phase the whole network is fine-tuned with a smaller

learning rate. The hyperparameters of the two phases are summarised in Table 5.2. Training used automatic mixed precision on a Tesla P100 GPU to reduce time and memory, and the exact augmentation transforms are listed in Appendix A.

Setting	Phase 1 (feature extraction)	Phase 2 (fine-tuning)
Trainable parameters	Classification head only	Entire network
Optimiser	Adam	AdamW
Learning rate	1×10^{-3}	1×10^{-4}
Weight decay	—	1×10^{-4}
Epochs	10	12
Batch size	64	64

Table 5.2: Two-phase transfer-learning hyperparameters.

The third element addresses class imbalance. A weighted random sampler equalises the two classes within each minibatch, and a class-weighted cross-entropy loss further penalises errors on the minority live class. Together these measures ensure that the optimisation does not collapse onto the majority class.

5.1.3 Result

The training behaviour confirms the value of the two-phase strategy. During the first phase, with the backbone frozen, the training accuracy plateaus near 82.7%, because the generic ImageNet features are not yet specialised to spoofing cues. Once the network is unfrozen in the second phase, the training accuracy rises rapidly and reaches 99.86% by the final epoch. This progression is plotted in Figure 5.1. The high training accuracy by itself is not evidence of generalisation; the cross-dataset test results that establish generalisation are reported in Section 5.3.

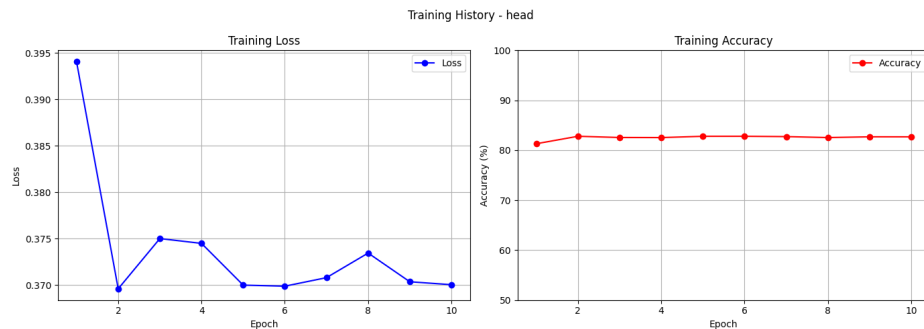


Figure 5.1: Training loss and accuracy across the feature-extraction and fine-tuning phases.

5.2 Real-time serving design

5.2.1 Problem

A trained model is only useful if it can be queried interactively. The serving problem has two aspects: latency, since the verdict must be returned quickly enough for a live interaction, and resilience, since a streaming connection over an imperfect network will inevitably encounter dropped connections and occasional malformed frames that must not bring down the service.

5.2.2 Solution

Latency is addressed by streaming over a persistent WebSocket and by limiting the client to one frame every 500 milliseconds, which bounds the load while keeping the display responsive. On the server, the asynchronous FastAPI endpoint decodes each frame, runs the model under a no-gradient context, and pushes the verdict back over the same connection, avoiding the per-request overhead that a request–response design would incur for every frame.

Resilience is addressed on both sides. On the server, every incoming message is validated, and a malformed frame, an unknown message type, or a model error produces a descriptive error response while the connection stays open, so that one bad frame cannot interrupt the stream. On the client, the connection is re-established automatically after a loss, using an exponential back-off that waits progressively longer between attempts, and a periodic keep-alive is sent while the connection is idle. The verdict is decoupled from the capture loop, so that the interface continues to display the most recent result even while the next frame is in flight.

5.2.3 Result

The serving design meets the real-time requirement. On the development machine the demonstrator processes frames at the intended rate of about two per second with a verdict latency below one second, and the connection recovers transparently from deliberate interruptions during testing. The clean separation between the web layer and the model layer also means that an improved model can be deployed by replacing a single weights file, without any change to the serving code.

5.3 Security evaluation and analysis

5.3.1 Problem

The value of a liveness system is ultimately determined by how well it resists attacks while not obstructing genuine users. A meaningful evaluation must therefore measure performance on data that differ from the training data and must report the result in security terms, not merely as accuracy.

5.3.2 Solution

The model is evaluated on a held-out cross-dataset test set of 83,514 images drawn from the test partitions of the same three corpora, whose composition is given in Table 5.3. The test set is deliberately weighted towards spoofs, which represent about 83% of the images, reflecting an attack-heavy scenario and making per-class metrics essential. The metrics of Section 3.6 are computed from the confusion matrix obtained on this set.

Source	Images
CelebA-Spoof (balanced test)	7,200
CASIA-FASD (test split)	65,786
LCC-FASD (development + evaluation)	10,528
Total (Spoof 69,067 / Live 14,447)	83,514

Table 5.3: Composition of the cross-dataset test set.

5.3.3 Result

Table 5.4 reports the classification metrics. The model attains an accuracy of 95.83%, a recall of 97.00%, a precision of 82.13%, and an F1-score of 88.95% on data unseen during training. The gap between the near-perfect training accuracy and the 95.83% test accuracy quantifies the domain shift between datasets and represents the realistic, harder case that the cross-dataset design deliberately measures.

Metric	Value
Accuracy	95.83%
Precision	82.13%
Recall	97.00%
F1-score	88.95%

Table 5.4: Classification performance on the cross-dataset test set.

The per-class behaviour is summarised by the confusion matrix in Table 5.5, whose counts are consistent with the metrics above and with the test-set class distribution; the corresponding heat-map produced by the notebook can be inserted as Figure 5.2. Of the 69,067 spoof images, about 3,049 were wrongly accepted as live, and of the 14,447 live images, about 434 were wrongly rejected.

	Predicted Spoof	Predicted Live
Actual Spoof (69,067)	66,018 (TN)	3,049 (FP)
Actual Live (14,447)	434 (FN)	14,013 (TP)

Table 5.5: Confusion matrix on the cross-dataset test set.

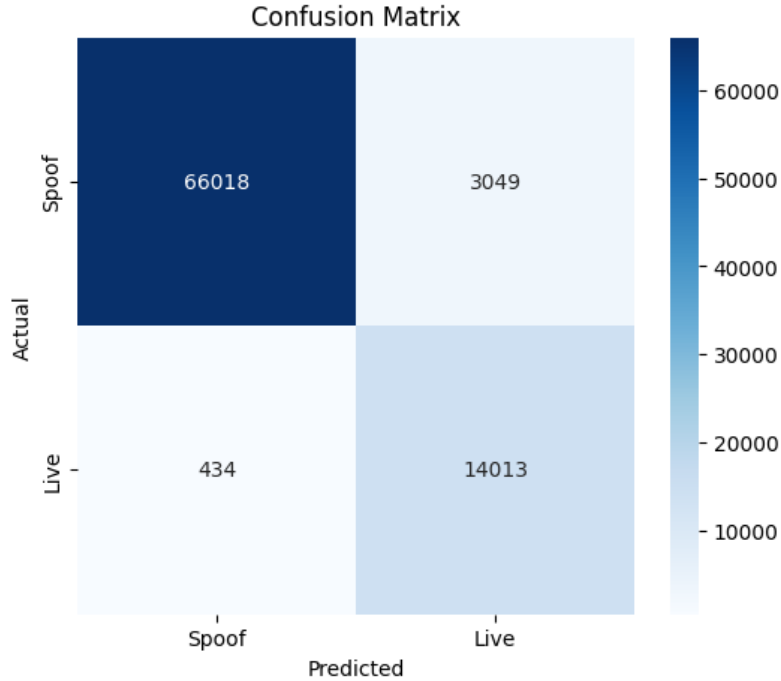


Figure 5.2: Confusion-matrix heat-map on the cross-dataset test set.

Translating these counts into the biometric security metrics of Equations (3.5) to (3.7) gives the values in Table 5.6. The false acceptance rate is about 4.41%, the false rejection rate about 3.00%, and the average classification error rate about 3.71%.

Security metric	Meaning	Value
FAR / APCER	Spoofs accepted as live	4.41%
FRR / BPCER	Live users rejected	3.00%
ACER	Average of the two	3.71%

Table 5.6: Security metrics derived from the confusion matrix.

These figures admit a clear interpretation. The low false rejection rate of about 3% means that genuine users are seldom turned away, so the system is convenient to use. The false acceptance rate of about 4.4% is the security-critical figure: roughly one spoof presentation in twenty-three is wrongly accepted. For a low-risk, convenience-oriented application this is acceptable, but for a high-assurance setting it is too permissive for the model to be used on its own. The lower precision

compared with recall is explained by the same false accepts, which are numerous in absolute terms on this attack-heavy test set.

A practical strength of the design is that this operating point can be adjusted. Because the serving code selects the class with the higher probability, it implicitly uses a decision threshold of one half. Requiring instead that the live probability exceed a higher threshold would reduce the false acceptance rate at the cost of a higher false rejection rate, allowing the operating point to be calibrated to the risk profile of a given application. The residual vulnerability that remains, a single passive classifier can still be deceived by a sufficiently high-quality replay, motivates the active-liveness extension proposed in the next chapter.

This chapter has presented the cross-dataset training pipeline, the resilient real-time serving design, and the security evaluation of the system, the last of which establishes both its usability and the limits of its protection. The next chapter draws overall conclusions and sets out future work.

CHAPTER 6. CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project set out to build a lightweight, real-time face liveness detection system capable of distinguishing genuine faces from common two-dimensional presentation attacks using only an ordinary webcam, and to evaluate that system in biometric security terms. All of these goals were met. A cross-dataset training pipeline was developed that fine-tunes an EfficientNet-B3 network on a mixture of the CelebA-Spoof, CASIA-FASD, and LCC-FASD datasets, with class imbalance handled by weighted sampling and a class-weighted loss. The trained model was integrated into a client–server application in which a React frontend streams webcam frames over a WebSocket to an asynchronous FastAPI backend that returns a live-or-spoof verdict in real time.

Measured on a held-out cross-dataset test set of 83,514 images, the model reached an accuracy of 95.83%, a recall of 97.0%, and an F1-score of 88.9%, corresponding to a false acceptance rate of about 4.4% and a false rejection rate of about 3.0%. Compared with classical methods based on hand-crafted texture features, the deep cross-dataset approach generalises noticeably better across capture conditions, while remaining far cheaper to deploy than hardware-based solutions that require depth or infrared sensors. Compared with commercial systems, the present work is of course more limited in robustness and breadth of attack coverage, but it demonstrates the full pipeline end to end and reports its performance honestly.

The project also clarified what was not achieved. The system relies on a single passive classifier and does not yet implement the active, blink-based liveness check that was part of the original design; it performs no explicit face detection or cropping before inference, feeding the whole frame to the model; and it uses a fixed decision threshold rather than one calibrated to a target error rate. A few minor robustness defects were also identified during testing, notably that the client keep-alive message is treated by the server as an unrecognised message type, and that a server-side retry delay is not awaited correctly. None of these prevents the system from working, but each represents an opportunity for improvement. The principal lesson drawn from the work is that cross-dataset training, together with disciplined handling of class imbalance, is decisive for obtaining a model that generalises, and that an honest, security-oriented evaluation is as important as a high headline accuracy.

6.2 Future work

Future work falls into two groups: completing and hardening the functions already built, and extending the system with new capabilities.

The first group concerns consolidation. The minor defects noted above should be corrected by handling the keep-alive message explicitly on the server and by awaiting the retry delay. An explicit face detector should be added so that the model receives a cropped face region that matches the conditions under which it was trained, which is expected to improve accuracy. The decision threshold should be calibrated on a validation set to a chosen operating point, and the evaluation should be strengthened by reporting error rates per attack type and by adding a full error-rate curve and an equal-error-rate figure.

The second group concerns new capabilities. The most important is the addition of active liveness detection based on the eye aspect ratio [14], computed from facial landmarks, so that a confirmed blink is required within a session; combining this temporal cue with the spatial decision of the convolutional model would substantially raise the cost of a successful attack and realise the hybrid design that was originally envisaged. Beyond this, the system could incorporate richer cues such as remote photoplethysmography or optical-flow micro-motion, could be tested against entirely unseen datasets and devices for a fairer estimate of generalisation, and could be extended with verification logging and rate limiting to move it closer to a production-ready service.

REFERENCE

- [1] Z. Boulkenafet, J. Komulainen, and A. Hadid, “Face anti-spoofing based on color texture analysis,” in *2015 IEEE International Conference on Image Processing (ICIP)*, IEEE, 2015, pp. 2636–2640.
- [2] Y. Liu, A. Jourabloo, and X. Liu, “Learning deep models for face anti-spoofing: Binary or auxiliary supervision,” in *2018 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 389–398.
- [3] Z. Yu, Y. Qin, X. Li, C. Zhao, Z. Lei, and G. Zhao, “Deep learning for face anti-spoofing: A survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, vol. 45, no. 5, pp. 5609–5631, 2022.
- [4] M. Tan and Q. Le, “Efficientnet: Rethinking model scaling for convolutional neural networks,” in *International Conference on Machine Learning (ICML)*, PMLR, 2019, pp. 6105–6114.
- [5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “Mobilenetv2: Inverted residuals and linear bottlenecks,” in *2018 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.
- [6] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *2009 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, 2009, pp. 248–255.
- [7] PyTorch Foundation, *PyTorch documentation*, <https://pytorch.org>, Accessed: 2026-06-10, 2024.
- [8] S. Ramírez, *FastAPI framework documentation*, <https://fastapi.tiangolo.com>, Accessed: 2026-06-10, 2024.
- [9] Meta Open Source, *React – a javascript library for building user interfaces*, <https://react.dev>, Accessed: 2026-06-10, 2024.
- [10] International Organization for Standardization, *ISO/IEC 30107-3:2017 – information technology – biometric presentation attack detection – part 3: Testing and reporting*, Standard, 2017.
- [11] Y. Zhang et al., “Celeba-spoof: Large-scale face anti-spoofing dataset with rich annotations,” in *European Conference on Computer Vision (ECCV)*, Springer, 2020, pp. 70–85.
- [12] Z. Zhang, J. Yan, S. Liu, Z. Lei, D. Yi, and S. Z. Li, “A face antispoofing database with diverse attacks,” in *2012 5th IAPR International Conference on Biometrics (ICB)*, IEEE, 2012, pp. 26–31.

- [13] D. Timoshenko, K. Simonchik, V. Shutov, P. Zhelezneva, and V. Grishkin, “Large crowdcollected facial anti-spoofing dataset,” in *2019 Computer Science and Information Technologies (CSIT)*, IEEE, 2019, pp. 123–126.
- [14] T. Soukupová and J. Čech, “Real-time eye blink detection using facial landmarks,” in *21st Computer Vision Winter Workshop*, 2016, pp. 1–8.

APPENDIX

A. TRAINING CONFIGURATION AND DATASET DETAILS

This appendix collects the detailed training configuration and the per-epoch logs that support the results reported in Chapter 5.

A.1 Preprocessing and augmentation

All images are converted to three-channel RGB, resized to 224×224 pixels, and normalised with the ImageNet statistics, namely a mean of $[0.485, 0.456, 0.406]$ and a standard deviation of $[0.229, 0.224, 0.225]$. During training the following augmentations are applied in order: a random resized crop to 224 pixels with a scale range of 0.75 to 1.0; a random horizontal flip with probability 0.5; a random rotation of up to 25 degrees; a colour jitter with brightness 0.3, contrast 0.3, saturation 0.2, and hue 0.1; a Gaussian blur with kernel size three applied with probability 0.4; and a random erasing with probability 0.4 and a scale range of 0.02 to 0.2. At inference time only the resize and normalisation are applied.

A.2 Training environment

The model was trained in a Kaggle notebook on a single NVIDIA Tesla P100 GPU with 16 gigabytes of memory, using PyTorch 2.6 with CUDA 12.4. Training used automatic mixed precision, a batch size of 64, and a weighted random sampler over the 137,140 training images, giving 2,142 minibatches per epoch.

A.3 Per-epoch training log

Table A.1 and Table A.2 list the training loss and accuracy for each epoch of the two phases. In the first phase the backbone is frozen, so accuracy plateaus near 82.7%; in the second phase the whole network is fine-tuned and accuracy rises to 99.86%.

Epoch	Loss	Accuracy (%)
1	0.3941	81.31
2	0.3696	82.80
3	0.3750	82.55
4	0.3745	82.54
5	0.3700	82.79
6	0.3699	82.79
7	0.3708	82.73
8	0.3735	82.54
9	0.3704	82.69
10	0.3700	82.68

Table A.1: Phase 1 (feature extraction) per-epoch training log.

Epoch	Loss	Accuracy (%)
1	0.0613	97.54
2	0.0189	99.29
3	0.0114	99.57
4	0.0097	99.66
5	0.0068	99.76
6	0.0066	99.76
7	0.0058	99.80
8	0.0045	99.83
9	0.0045	99.84
10	0.0044	99.85
11	0.0032	99.89
12	0.0041	99.86

Table A.2: Phase 2 (fine-tuning) per-epoch training log.

B. API AND WEBSOCKET MESSAGE SCHEMAS

This appendix specifies the application programming interfaces exposed by the backend, both the request–response endpoint and the WebSocket message schemas referenced in Chapter 4.

B.1 Request–response endpoint

The single-image endpoint is reached by a POST request to `/api/anti-spoof/predict` with the image supplied as multipart form data. On success it returns a JSON object containing the predicted label, the confidence, and the per-class probabilities. If the uploaded file is not an image the server returns a client-error response, and if the model is not loaded it returns a service-unavailable response.

B.2 WebSocket message schemas

The real-time endpoint is reached at `/api/anti-spoof/ws`. The client sends a prediction request for each captured frame, and the server replies with either a prediction or an error. Listing B.1 shows the three message schemas.

```
1 // Client -> Server
2 { "type": "predict", "image": "<base64-encoded JPEG>", "timestamp
   ": 1683350400000 }
3
4 // Server -> Client (success)
5 { "type": "prediction", "prediction": "Live",
6   "confidence": 0.95,
7   "probabilities": { "Spoof": 0.05, "Live": 0.95 },
8   "timestamp": 1683350400000 }
9
10 // Server -> Client (failure)
11 { "type": "error", "message": "<description>", "timestamp":
    1683350400000 }
```

Listing B.1: WebSocket message schemas (client request, server prediction, server error).

The image field carries the frame as a base64-encoded JPEG with the data-URI prefix removed. The timestamp is echoed back so that the client can associate each verdict with the frame that produced it. The server validates every message and returns an error object, rather than closing the connection, when the JSON is malformed, the message type is unrecognised, the image field is missing, the base64 payload cannot be decoded, or the model fails.